

Code Review Report

CMPT 276 Assignment 4

Reviewer: Lucas Nguyen Reviewee: Megan Chau (mhc23)

March 31, 2026

Overview

For this assignment I reviewed code written by Megan Chau (mhc23). I looked at three files that she authored: `SettingsScreen.java`, `InstructionsScreen.java`, and `EndScreen.java` on the `Assignment4` branch. Authorship was confirmed with `git blame`. I found four code smells, which are described below.

1 Smell 1: Code Duplication Across Two Screen Classes

Files: `SettingsScreen.java` and `InstructionsScreen.java` **Commit:** 77e81fd

In commit 77e81fd, Megan added the same audio setup block to both `SettingsScreen.show()` and `InstructionsScreen.show()`. In both files the code reads:

```
escSound = Gdx.audio.newSound(Gdx.files.internal("audio/escSound.mp3"));
toggleSound = Gdx.audio.newSound(Gdx.files.internal("audio/toggleSound.mp3"));
if (SettingsScreen.soundOn) {
    toggleSound.play(0.8f);
}
```

These four lines are identical in both classes, down to the file paths and the volume value. Duplicated code is a maintenance problem because any future change to this logic, such as adjusting the volume or swapping out a sound file, has to be made in two places. It is easy for the copies to drift out of sync over time. A better approach would be to extract the shared audio setup into a helper method or a common base class so the logic only lives in one place.

2 Smell 2: Data Clumps for the Toggle Button Hit Zone

File: `SettingsScreen.java`, lines 27 to 28 **Commit:** 5062325

The clickable region for the sound toggle button is defined as four separate float constants:

```
private static final float TOGGLE_X1 = 500f, TOGGLE_X2 = 750f;
private static final float TOGGLE_Y1 = 345f, TOGGLE_Y2 = 445f;
```

Whenever this region needs to be checked, all four constants travel together as a group. This is a data clump because the four values only have meaning as a unit and always appear together in usage. LibGDX already provides a `Rectangle` class that can hold a region and exposes a built-in `contains()` method, which would replace the four separate constants and remove the need to pass them individually every time the hit zone is tested.

3 Smell 3: Constants Updated Without Updating the Accompanying Comment

File: `EndScreen.java`, lines 30 to 33 **Commit:** 29afa63

In `EndScreen`, there is a comment above the score digit constants that explains how the values were calculated:

```
// 5 digits at DIGIT_W each + 4 gaps = 670, so gap = (670 - 5*115) / 4 = 24.
private static final float DIGIT_W = 150f;
private static final float DIGIT_H = 140f;
private static final float DIGIT_X = 348f;
```

In commit 29afa63, Megan updated `DIGIT.W` to 150f and the surrounding constants, but the comment above still references `DIGIT.W = 115` and a gap of 24. Neither number matches the actual constants anymore. A developer reading this file has no way to know whether to trust the comment or the code. This is an example of poorly structured code where stale documentation actively misleads rather than helps. The comment should have been

updated at the same time the constants were changed.

4 Smell 4: Sound Objects Not Disposed

Files: `SettingsScreen.java` and `InstructionsScreen.java` **Commit:** 77e81fd

In commit 77e81fd, Megan added `toggleSound` and `escSound` fields to both `SettingsScreen` and `InstructionsScreen`, loading them in `show()`:

```
toggleSound = Gdx.audio.newSound(Gdx.files.internal("audio/toggleSound.mp3"));
escSound    = Gdx.audio.newSound(Gdx.files.internal("audio/escSound.mp3"));
```

However, neither `dispose()` method was updated to release these objects. Both methods only dispose the textures and the `SpriteBatch`, leaving the `Sound` instances uncleaned:

```
@Override
public void dispose() {
    settingsOnTexture.dispose(); // toggleSound and escSound are never disposed
    settingsOffTexture.dispose();
    batch.dispose();
}
```

In LibGDX, `Sound` objects hold native audio resources that are not reclaimed by the garbage collector. Every time the player navigates to `SettingsScreen` or `InstructionsScreen` and back, new `Sound` instances are created in `show()` without the previous ones ever being freed. This is a resource leak. The fix is to call `toggleSound.dispose()` and `escSound.dispose()` inside each `dispose()` method.

Refactoring Based on Peer Review of My Code

Tanveer Muntaseer reviewed my file `CinematicBars.java` on the `Assignment4` branch and identified five code smells. Below I describe each smell, the fix I made, and the corresponding commit.

5 Smell 1: Abbreviated Variable Name in Constructor — commit 5e4f6bd

The local variable holding the temporary pixel buffer was named `pm`. The name is a two-letter abbreviation with no clear meaning to a reader who is not already familiar with the LibGDX `Pixmap` API. I renamed it to `pixmap` so the constructor is self-explanatory.

6 Smell 2: Redundant `currentHeight` Field — commit 26b007e

The class stored `private float currentHeight` alongside `timer` and `state`, even though the height is completely determined by those two fields. Keeping it as separate state meant every transition had to update both `timer` and `currentHeight` in sync. I removed the field and replaced all reads of it with a `private computeHeight()` method that derives the value on demand, giving the class a single source of truth for the animation progress.

7 Smell 3: Missing Javadoc on Public Methods — commit 9d90cf9

None of the five public methods (`show`, `hide`, `update`, `render`, `isFullyVisible`) had Javadoc. A caller looking only at the signatures would not know that `update()` must be called every frame, that `show()/hide()` handle mid-animation reversal, or that `render()` requires an already-open `SpriteBatch` using the HUD projection matrix. I added a Javadoc block to each method covering its purpose and any non-obvious usage requirements.

8 Smell 4: Implicit `else` in `update()` — commit d571704

The second branch of the `if/else` in `update()` used a bare `else` that silently assumed the state was `DISAPPEARING`. If a new state were added to the enum later, the `else` body would run against it incorrectly with no warning. I changed it to `else if (state == State.DISAPPEARING)` so the assumption is explicit.

9 Smell 5: High Coupling to `GameScreen` Constants — commit 1546474

The constructor read `GameScreen.VIEW_WIDTH` and `GameScreen.VIEW_HEIGHT` directly, creating an unnecessary compile-time dependency on `GameScreen` and making the class impossible to reuse on any screen with different dimensions. I changed the constructor to accept `screenWidth` and `screenHeight` as parameters and updated the call site in `GameScreen.show()` to pass `VIEW_WIDTH` and `VIEW_HEIGHT` explicitly.